

SKILL_06

1. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 history.c  
int count = 0;  
  
while(1)  
{  
    printf("Enter Command (history/exit): ");  
    fgets(command, MAX_CMD, stdin);  
  
    command[strcspn(command, "\n")] = '\0';  
  
    if(strcmp(command, "exit") == 0)  
        break;  
  
    if(strcmp(command, "history") == 0)  
    {  
        printf("\nCommand History:\n");  
        for(int i = 0; i < count; i++)  
            printf("%d. %s\n", i + 1, history[i]);  
        continue;  
    }  
  
    if(count < MAX_HISTORY)  
    {  
        strcpy(history[count], command);  
        count++;  
    }  
    else  
    {  
        for(int i = 1; i < MAX_HISTORY; i++)  
            strcpy(history[i - 1], history[i]);  
  
        strcpy(history[MAX_HISTORY - 1], command);  
    }  
}  
  
return 0;  
}
```

RESULT:

```

root@SRIVALLI:~# nano history.c
root@SRIVALLI:~# gcc history.c -o history
root@SRIVALLI:~# ./history
Enter Command (history/exit): ls
pwd
date
whoaEnter Command (history/exit): Enter Command (history/exit): Enter Comman
d (history/exit): mi
clear
historyEnter Command (history/exit): Enter Command (history/exit):
Command History:
1. ls
2. pwd
3. date
4. whoami
5. clear
Enter Command (history/exit): exit

```

REPORT:

In this experiment, a command history management program was developed to store and maintain previously entered commands. A fixed-size history buffer was used to save user commands. When the buffer reached its maximum capacity, the oldest command was removed and the newest command was added. The program allows users to retrieve and display command history using the history command. This experiment helps in understanding command storage, history buffer management, capacity handling, command retrieval, and consistency maintenance in shell environments.

2. To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

root@SRIVALLI: ~
GNU nano 7.2 pipeline.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pipe(fd);

    pid_t pid1 = fork();

    if(pid1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("ls failed");
        exit(1);
    }

    pid_t pid2 = fork();

    if(pid2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc failed");
    }
}

```

CODE:

RESULT:

```

root@SRIVALLI:~# nano pipeline.c
root@SRIVALLI:~# gcc pipeline.c -o pipeline
root@SRIVALLI:~# ./pipeline
75
root@SRIVALLI:~# |

```

REPORT:

In this experiment, a pipeline structure was created using the pipe() system call to connect two processes. The first process executes the ls command and sends its output through the pipe, while the second process receives the data and executes the wc -l command to count the number of entries. The execution order is maintained using fork(), and process communication is achieved through the pipe. This experiment demonstrates how pipelines connect multiple processes, transfer data between commands, and maintain proper execution flow in Linux systems.

3. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

CODE:

```
GNU nano 7.2 pipe_commands.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    pid_t pid = fork();

    if(pid == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
}
```

RESULT:

```
root@SRIVALLI:~# nano pipe_commands.c
root@SRIVALLI:~# gcc pipe_commands.c -o pipe_commands
root@SRIVALLI:~# ./pipe_commands
78
root@SRIVALLI:~# ls -l | wc -l
78
```

REPORT:

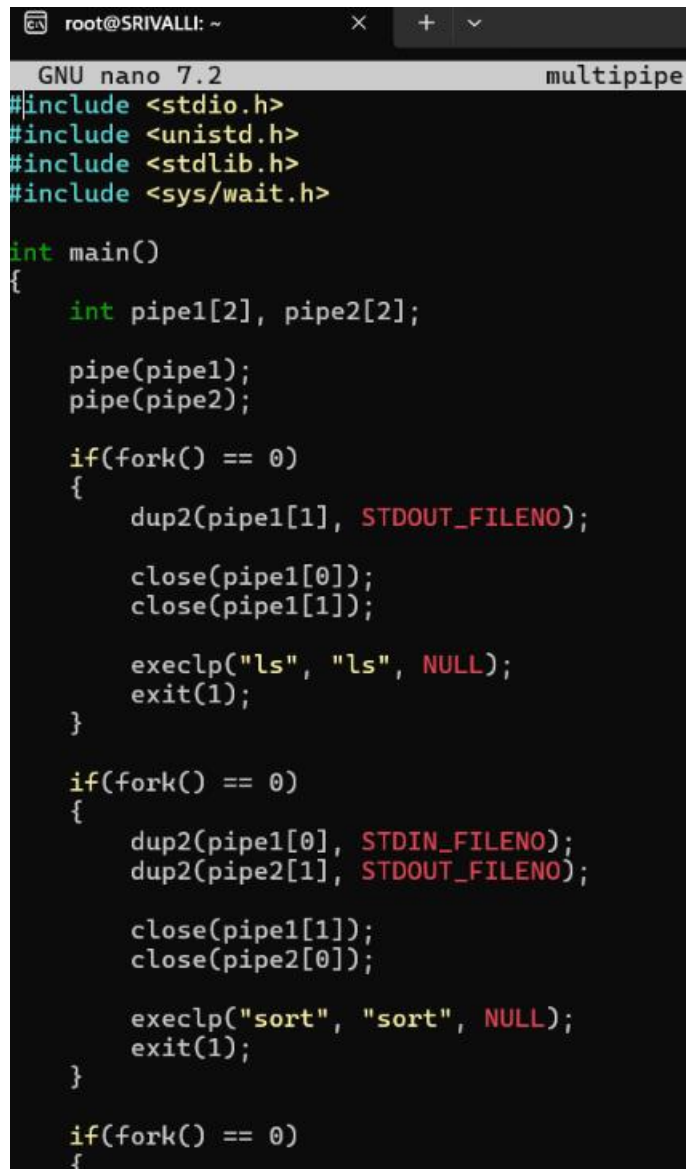
In this experiment, a pipe was created using the pipe() system call to establish communication between two processes. The output stream of the first process was redirected to the pipe using dup2(), and the second process received the data through its input stream. The commands were executed using execlp(), and unused file descriptors were closed to ensure proper resource management. Process synchronization was achieved through parent-child execution, and the data flow between commands was successfully

verified. This experiment helps in understanding pipes, stream redirection, process communication, and command execution in Linux systems.

4.

To Support Multiple Pipes, Manage Descriptor Arrays,
Launch Multiple Processes, Coordinate Execution, Cleanup
Resources, Test Long Pipelines

CODE:



```
root@SRIVALLI: ~  
GNU nano 7.2 multipipe  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
#include <sys/wait.h>  
  
int main()  
{  
    int pipe1[2], pipe2[2];  
  
    pipe(pipe1);  
    pipe(pipe2);  
  
    if(fork() == 0)  
    {  
        dup2(pipe1[1], STDOUT_FILENO);  
  
        close(pipe1[0]);  
        close(pipe1[1]);  
  
        execlp("ls", "ls", NULL);  
        exit(1);  
    }  
  
    if(fork() == 0)  
    {  
        dup2(pipe1[0], STDIN_FILENO);  
        dup2(pipe2[1], STDOUT_FILENO);  
  
        close(pipe1[1]);  
        close(pipe2[0]);  
  
        execlp("sort", "sort", NULL);  
        exit(1);  
    }  
  
    if(fork() == 0)  
    {
```

```

if(fork() == 0)
{
    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);

    close(pipe1[1]);
    close(pipe2[0]);

    execlp("sort", "sort", NULL);
    exit(1);
}

if(fork() == 0)
{
    dup2(pipe2[0], STDIN_FILENO);

    close(pipe2[1]);

    execlp("wc", "wc", "-l", NULL);
    exit(1);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

wait(NULL);
wait(NULL);
wait(NULL);

return 0;

```

RESULT:

```

root@SRIVALLI:~# nano multipipe.c
root@SRIVALLI:~# gcc multipipe.c -o multipipe
root@SRIVALLI:~# ./multipipe

```

REPORT:

In this experiment, multiple pipes were created to establish communication among several processes in a long pipeline. Two pipes were used to connect three commands: ls, sort, and wc -l. Each process was created using fork(), and input/output streams were redirected using dup2(). File descriptor arrays were managed carefully, and unused descriptors were closed to prevent resource leaks. The processes executed in a coordinated manner, passing data through the pipeline. This experiment demonstrates the implementation of long pipelines, inter-process communication, process coordination, and resource management in Linux systems.